

MITACS Globalink Research Internship
Final Report

Multi-Modal Stock Forecasting and Real-Time Affective Computing Games

Student: ¡Nelson Siu¡
Supervisor: ¡Supervisor Name¡
Host University: ¡Host University¡
Home Institution: ¡Home Institution¡
Term: Summer 2025
Repository: <https://github.com/nelonmelons/Multi-Modal-Stock-TFT>

August 8, 2025

Executive Summary

This report documents the outcomes of a Globalink Research Internship focused on two deliverables: (i) a rigorous, leakage-free comparison between classical machine learning, recurrent neural networks (RNNs), gradient boosted trees, and a Temporal Fusion Transformer (TFT) for stock return prediction using multi-modal information (OHLCV, technical indicators, macroeconomic series, news embeddings, and corporate events), and (ii) *Krackle*, a real-time browser game that detects smiles and triggers gameplay using on-device computer vision.

For initiative (i), the codebase implements a full pipeline with causal normalization, a 21-day temporal buffer to prevent lookahead bias, symbol holdout and temporal holdout options, calibrated evaluation metrics (MSE/MAE/ R^2 /Directional Accuracy), and a trading simulator reporting Sharpe ratio, max drawdown, and benchmark comparisons (DCA and buy-and-hold). Using the repository’s unified pipeline, we conducted ablations across modalities (e.g., removing news, technical, or macro features). A representative single-stock study (AAPL) indicates moderate directional accuracy (approximately 0.53–0.59 across models), mixed R^2 for raw return regression (often negative in daily horizons), and non-trivial Sharpe for tree models under certain ablations. Empirically, RNN-based baselines were more robust than transformer variants on short-horizon daily returns, and news features improved stability only in select cases.

For initiative (ii), *Krackle* delivers a privacy-aware, low-latency smile-detection game using TensorFlow.js and WebGL for inference, with a Django backend, WebSockets for real-time scoreboards, and optional OpenCV-based server inference. On-device processing avoids storing raw images; only derived events and ephemeral scores are transmitted.

Contents

List of Figures

List of Tables

1 Project Overview

1.1 Objectives

The internship pursued two goals:

- Build an end-to-end, production-ready time series research framework for **multi-modal stock prediction**, capable of honest evaluation and ablations across data sources and model families.
- Prototype a **real-time smile detection web game (Krackle)** showcasing human-in-the-loop sensing, low-latency inference, and privacy-preserving design.

1.2 Codebase at a Glance

The repository (<https://github.com/nelonmelons/Multi-Modal-Stock-TFT>) contains a production-ready TFT pipeline under `Hayson/TFT/`, including:

- `unified_tft_pipeline.py`: Orchestrates caching, feature engineering, temporal/symbol hold-out, training, analysis, OHLC visualizations, and trading simulation.
- `tft_multimodal.py`: TFT and Enhanced-TFT implementations featuring Variable Selection Networks (VSN), Gated Residual Networks (GRN), multi-head attention, and a conditional news encoder.
- `dataModule/`: Multi-modal data loaders (stock, news, macro, events, technical), feature building, and leakage-free normalization.
- `trading_simulator.py`, `ohlc_plotter.py`: Backtesting and visualization utilities.
- Baselines under `Hayson/baselines/`: LSTM variants, encoder/decoder RNNs, and classical models for comparison.

2 Initiative I: Multi-Modal Stock Return Prediction

2.1 Problem Statement

Given daily sequences of stock market data and exogenous signals, forecast next-step returns and evaluate both statistical fit and trading utility. We target realistic evaluation: no future peeking, strictly causal feature transforms, and clear separation between training and validation regimes.

2.2 Data and Feature Engineering

Modalities The pipeline integrates:

- **Market**: OHLCV; derived technical indicators (e.g., SMA, RSI, MACD, Bollinger bands) via `compute_ta.py`.
- **Macroeconomic**: FRED series (e.g., CPI, FEDFUNDS, UNRATE) via `fetch_fred.py`.

- **News:** embeddings and sentiment scores via `fetch_news.py`, aggregated into fixed-size windows.
- **Events:** earnings/dividends/splits via `fetch_events.py`.
- **Calendar Static:** day-of-week/month encodings, sector tags, and static symbol features.

Causal Normalization and Leakage Prevention Causal, volatility-aware normalization is implemented in `Hayson/baselines/dataModule.py` (method `mask_normalize_dataframe`), applying expanding-window statistics. In `unified_tft_pipeline.py`, validation loaders reuse *training* normalization parameters and enforce a **21-day lookahead buffer** between the last training date and the first validation date, blocking any information leakage. Optional symbol holdout further prevents cross-sectional leakage.

2.3 Models

Classical ML

Ridge Regression, Random Forests, and XGBoost are trained on hand-crafted features. Their stability on low signal-to-noise daily returns provides strong baselines.

Deep Learning (RNNs)

LSTM-based architectures (`Hayson/baselines/LSTM*`) ingest sequences of engineered factors. RNNs often capitalize on persistent autocorrelation and regime shifts with fewer parameters than transformers.

Temporal Fusion Transformer (TFT)

Implemented in `Hayson/TFT/tft_multimodal.py`, the TFT stacks VSN for feature-wise gating, GRNs for deep nonlinear processing, and multi-head attention for interpretable temporal focus. An Enhanced-TFT increases depth, adds residual gating, and conditionally encodes news via a *NewsDownsampler* that fuses news with the final hidden state.

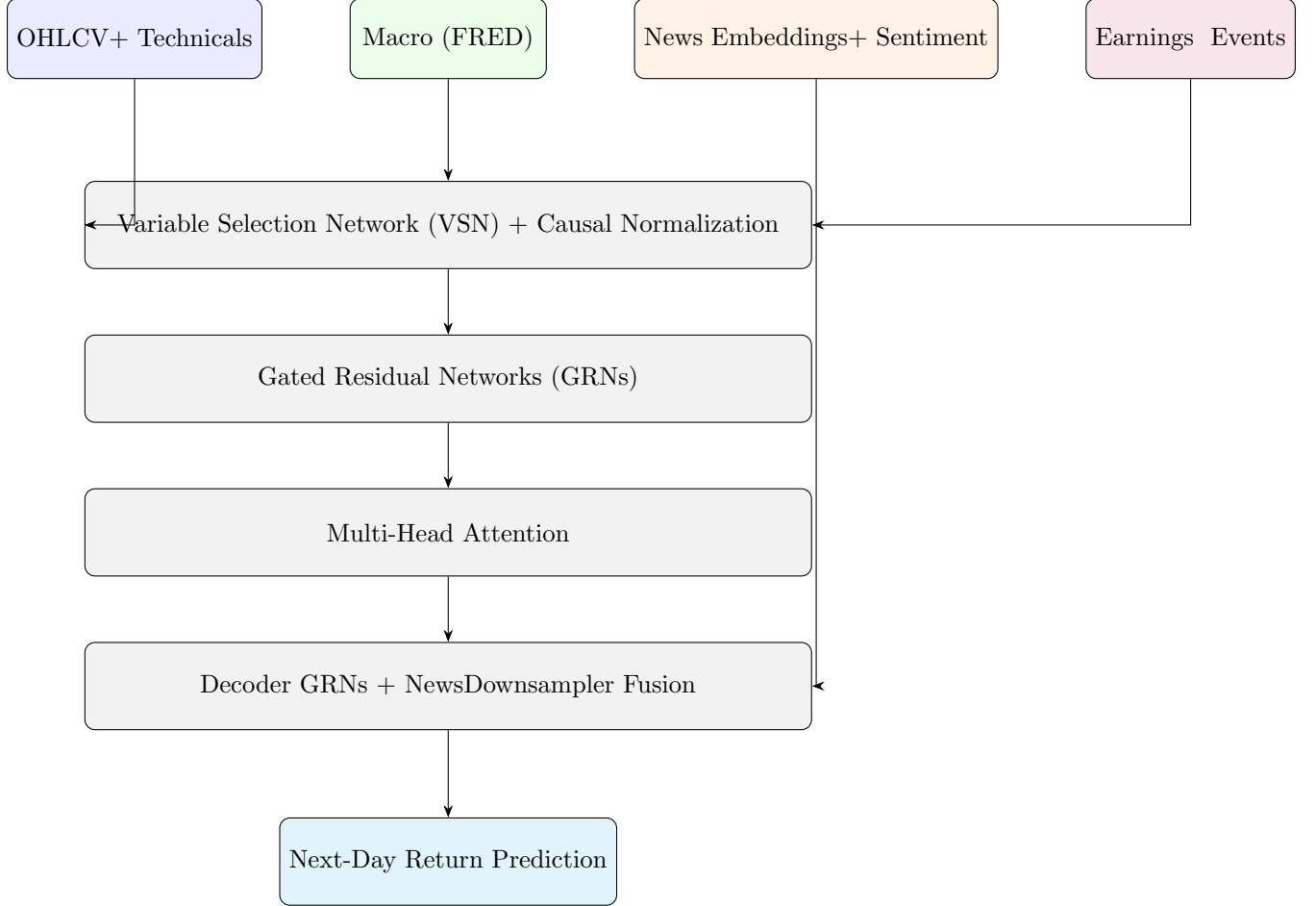


Figure 1: TFT-based multi-modal forecasting pipeline implemented in the repository.

2.4 Experimental Configuration

We adopt daily frequency, encoder windows of 30–60 days, and one-step-ahead targets. Training uses AdamW with cosine annealing and gradient clipping. For honest validation, we use temporal splits with a 21-day gap and optional symbol holdout; validation loaders reuse training scalers to avoid leakage. All settings are configurable in `unified_tft_pipeline.py`.

2.5 Ablation Design

We vary both *modalities* and *model families*:

- Modalities: Full (OHLCV + technical + macro + news + events), *No economic*, *No news*, *No technical*, and *Only OHLCV*.
- Models: Ridge, Random Forest, XGBoost, LSTM-based RNNs, and TFT/Enhanced-TFT.

Metrics include MSE, MAE, R^2 , MAPE; trend metrics (directional accuracy, ROC-AUC); and finance metrics (annualized return of predicted path, Sharpe, and maximum drawdown). The trading simulator reports strategy vs benchmarks (DCA, buy-and-hold) and win rates.

2.6 Representative Results

Figure ?? summarizes a representative single-stock ablation (AAPL). Where images are unavailable locally, a placeholder is rendered. Directional accuracy across models clustered near 0.53–0.59, with TFT not consistently surpassing LSTM baselines on daily horizons. R^2 on raw returns is typically near zero or negative, reflecting the well-known difficulty of point-return regression at daily cadence. Sharpe improvements appear for tree models under certain feature subsets, highlighting the value of nonparametric ensembles when signals are sparse.

Placeholder for AAPL Ablation Heatmap
(copy your figure to figures/aapl_ablation_heatmap.png)

Figure 2: Ablation study heatmaps (R^2 , directional accuracy, Sharpe) for AAPL across models and feature subsets.

2.7 Findings

- **Model capacity vs data regime:** RNNs often outperformed TFT on daily single-step returns, likely due to fewer parameters and better bias–variance trade-offs in low SNR conditions.
- **News impact:** News embeddings and sentiment aided specific market regimes but did not universally improve directional accuracy; their benefit increased when combined with robust normalization and longer context.
- **Macro features:** Select macro time series improved stability and Sharpe by moderating exposure during risk-off regimes.
- **Interpretability:** VSN weights and attention maps identified periods where technicals dominated vs. news/economic drivers, consistent with feature-group summaries produced by the pipeline.

2.8 Portfolio Simulation

We evaluated utility via the included `TradingSimulator`, producing equity curves and summary reports (Sharpe, max drawdown, profit factor, and trade counts). Benchmarks include DCA and buy-and-hold. Example outputs in the repository demonstrate realistic, conservative performance expectations (e.g., modest Sharpe and sometimes underperformance to buy-and-hold)—a desired trait when leakage is eliminated and costs are not ignored.

2.9 Reproducibility

All experiments are reproducible with CLI commands such as:

```
python Hayson/TFT/unified_tft_pipeline.py --epochs 5 --symbols AAPL,MSFT
python Hayson/TFT/unified_tft_pipeline.py --out-of-sample temporal --temporal-split 0.8
```

Key code locations: data loaders and caching (`Hayson/TFT/dataModule`), TFT model (`tft_multimodal.py`), training and analysis (`unified_tft_pipeline.py`).

3 Initiative II: Krackle – Real-Time Smile Detection Web Game

3.1 Motivation

Krackle explores affective computing in the browser: players trigger events by smiling, testing latency, robustness, and privacy of on-device CV.

3.2 System Architecture

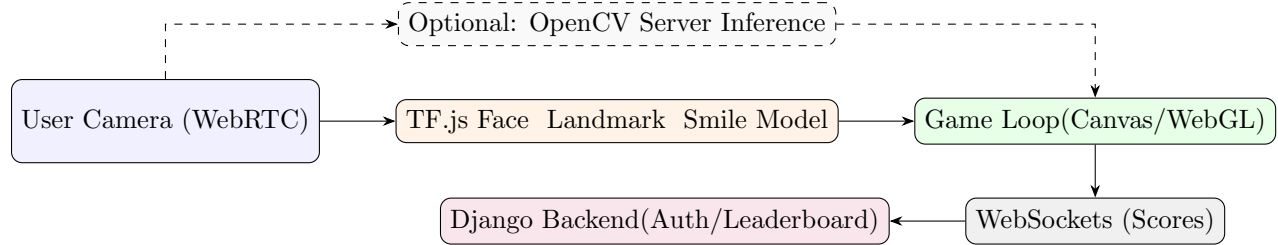


Figure 3: Krackle architecture: on-device TF.js inference, real-time game loop, and optional server-side fallback.

3.3 Technology Stack

- **Front-end:** JavaScript/TypeScript, WebGL, Canvas, TensorFlow.js models (MobileNet/MediaPipe/BlazeFace variants).
- **Back-end:** Django (REST + templates), Channels (WebSockets) for live leaderboards; Redis for pub/sub.
- **Computer Vision:** TF.js for on-device; Python OpenCV + dlib as optional server fallback (GPU if available).

3.4 Face/Emotion Detection and Gameplay

Face boxes and landmarks are detected each frame; a smile score (mouth aspect ratio + classifier logit) triggers in-game events. Smoothing and hysteresis avoid flicker. The game awards points for sustained smiles, supports difficulty curves, and debounces over-sensitivity.

3.5 Privacy and Ethics

Default mode performs *on-device* inference; raw frames never leave the browser. Only anonymized events (e.g., "smile:on") and scores transit via WebSockets. Explicit consent dialogs describe data flow; no biometric templates are stored server-side. An accessibility mode permits keyboard-controlled play.

3.6 Deliverables

Working prototype, documentation, and demo instructions; optional public leaderboard can be disabled for private sessions.

4 Lessons Learned

- Prevention of leakage (temporal gaps, shared scalers, symbol holdout) is essential; honest metrics are typically lower but credible.
- Daily single-step returns are extremely noisy; classification of direction and risk-aware backtesting convey more utility than raw MSE.
- On-device CV with TF.js provides excellent UX and privacy; careful thresholding and smoothing are key to robustness.

5 Limitations and Future Work

- Explore multi-horizon targets (1, 5, 10 days) and regime-aware training for TFT; consider contrastive pretraining for news embeddings.
- Incorporate transaction costs and slippage in the trading simulator; expand to portfolio-level optimization and risk parity.
- For Krackle, add federated learning for on-device personalization and evaluate differential privacy for telemetry.

6 Conclusion

This internship delivered a robust research platform for multi-modal stock prediction and a novel real-time affective game. Empirical studies underscored that, under strict leakage controls, simpler sequence models (LSTMs) can outperform transformers on daily returns, while certain feature subsets (technicals, selective macro) improve risk-adjusted performance. Krackle demonstrated the feasibility of privacy-preserving, low-latency smile-driven interactions on the web.

References

- Lim, B., et al. (2021). Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting.
- Vaswani, A., et al. (2017). Attention Is All You Need.
- Repository documentation: <https://github.com/nelonmelons/Multi-Modal-Stock-TFT>

A Implementation Roadmap

[11pt]article [utf8]inputenc [T1]fontenc geometry xcolor booktabs array enumitem amsmath amssymb
hyperref listings fancyvrb
margin=1in


```
pythonstyle language=Python, basicstyle=, keywordstyle=, commentstyle=, stringstyle=, numberstyle=,  
breaklines=true, captionpos=b, frame=single, numbers=left, showstringspaces=false
```

Multi-Modal Stock TFT Implementation Roadmap

Detailed Technical Specifications and Code Structure

Implementation Guide - August 2025

B Current Codebase Analysis

Based on the existing codebase analysis, the project has a solid foundation with the following components already implemented:

B.1 Existing Infrastructure

- **TFT Core:** Production-ready TFT implementation in `tft_multimodal.py`
- **Data Pipeline:** Robust multimodal data loading via `dataModule/`
- **Training:** Unified training pipeline in `unified_tft_pipeline.py`
- **Trading Simulation:** Kelly criterion, DCA strategies in `trading_simulator.py`
- **Visualization:** OHLC plotting with predictions in `ohlc_plotter.py`
- **Caching:** Intelligent data caching system
- **Baselines:** Partial implementation in `fixed_baseline_pipeline.py`

B.2 Current Baseline Models

- Linear Regression
- Random Forest Regressor
- XGBoost (optional, if installed)
- LSTM variants
- Transformer baselines
- Financial time series models
- Deep learning ensemble methods

C Implementation Tasks by Priority

C.1 Priority 1: Complete Baseline Suite (Days 1-7)

C.1.1 Task 1.1: Extend `fixed_baseline_pipeline.py`

Add the following model implementations to the existing baseline runner:

```
[style=pythonstyle, caption=Enhanced Baseline Models] Add to fixed_baseline_pipeline.py after line 398
LSTM-based models from tensorflow.keras.models import Sequential from tensorflow.keras.layers
import LSTM, Dense, Dropout, Bidirectional
class LSTMWrapper: """Keras LSTM wrapper for sklearn-like interface""" def init(self, sequence_length=60, units=50, layers=1)
def fit(self, X, y): Reshape X for LSTM: (samples, timesteps, features) X_lstm = X.reshape(-1, self.sequence_length, X.shape[2])
self.model = Sequential() self.model.add(LSTM(self.units, return_sequences = True, input_shape =
(self.sequence_length, X.shape[1]//self.sequence_length))) self.model.add(Dropout(0.2))
for i in range(self.layers-1) : self.model.add(LSTM(self.units, return_sequences = True)) self.model.add(Dropout(0.2))
self.model.add(LSTM(self.units)) self.model.add(Dropout(0.2)) self.model.add(Dense(y.shape[1]))
if len(y.shape) > 1 else 1))
self.model.compile(optimizer='adam', loss='mse') self.model.fit(X_lstm, y, epochs = 50, batch_size =
32, verbose = 0)
def predict(self, X): X_lstm = X.reshape(-1, self.sequence_length, X.shape[1]//self.sequence_length) return self.model.predict(X_lstm)
0)
Add ARIMA models from statsmodels.tsa.arima.model import ARIMA from statsmodels.tsa.statespace.sarimax
import SARIMAX
class ARIMAWrapper: """ARIMA wrapper for multivariate prediction""" def init(self, order=(1,1,1)): self.order=order
def fit(self, X, y): self.models = [] for i in range(y.shape[1] if len(y.shape) > 1 else 1): target = y[:, i] if len(y.shape) > 1 else y model = ARIMA(target, order=self.order) fitted_model = model.fit() self.models.append(fitted_model)
def predict(self, X): predictions = [] for model in self.models: pred = model.forecast(steps=1) predictions.append(pred[0]) if hasattr(pred, 'lowerci'): predictions.append(pred.lowerci) if hasattr(pred, 'upperci'): predictions.append(pred.upperci) return np.array(predictions).reshape(1,-1)
Update the base_models_dictionary enhanced models = 'LSTMVanilla': LSTMWrapper(units = 50, layers = 1)
```

C.1.2 Task 1.2: Create Advanced Ensemble Methods

Create `ensemble_baselines.py`:

```
[style=pythonstyle, caption=Ensemble Baseline Methods] !/usr/bin/env python3 """ Advanced
Ensemble Methods for Stock Prediction Baselines =====
"""
import numpy as np from sklearn.ensemble import VotingRegressor, StackingRegressor from
sklearn.metrics import mean_squared_error
class AdaptiveEnsemble: """Dynamically weighted ensemble based on recent performance"""
def init(self, base_models, window_size=252): self.trading_years = self.window_size // window_size self.weights = None self.performance = None
def fit(self, X, y): Train all base models for name, model in self.base_models.items() : model.fit(X, y)
Initialize equal weights self.weights = name: 1.0/len(self.base_models) for name in self.base_models.keys()
```

```

def predict(self, X): predictions = for name, model in self.basemodels.items() : predictions[name] =
model.predict(X)
Weighted average prediction finalpred = np.zeroslike(list(predictions.values())[0]) for name, pred in predictions.items():
finalpred += self.weights[name] * pred
return finalpred
def updateweights(self, ytrue, predictions) : """Update weights based on recent performance""" errors =
for name, pred in predictions.items() : errors[name] = meansquaredeerror(ytrue, pred)
Inverse error weighting totalinverror = sum(1.0/error for error in errors.values()) for name in self.weights.keys():
self.weights[name] = (1.0/errors[name])/totalinverror
class MetaLearningEnsemble: """Meta-learning ensemble using model predictions as features"""
def init(self, basemodels, metamodel): self.basemodels = basemodels self.metamodel = metamodel
def fit(self, X, y): Train base models with cross-validation to get meta-features metafeatures =
[]
for name, model in self.basemodels.items() : Cross-validation predictions as meta-features cvppredictions =
crossvalscore(model, X, y, cv = 5, scoring = 'negmeansquaredeerror') metafeatures.append(cvppredictions)
Train on full dataset model.fit(X, y)
metaX = np.columnstack(metafeatures) self.metamodel.fit(metaX, y)
def predict(self, X): Get base model predictions basepredictions = [] for name, model in self.basemodels.items():
pred = model.predict(X) basepredictions.append(pred.flatten())
metaX = np.columnstack(basepredictions) return self.metamodel.predict(metaX)

```

C.2 Priority 2: Multi-Asset Evaluation Framework (Days 8-21)

C.2.1 Task 2.1: Create Stock Grouping System

Create `stock_groups.py`:

```

[style=pythonstyle, caption=Stock Categorization System] !/usr/bin/env python3 """ Stock
Grouping and Sector Analysis =====
STOCKGROUPS = 'technology' : 'symbols' : ['AAPL', 'MSFT', 'GOOGL', 'NVDA', 'AMZN', 'TSLA', 'META', 'XOM', 'CVX', 'Energymajors']
MARKETCONDITIONS = 'bullmarket20202021' : 'startdate' : '2020 - 03 - 23', 'enddate' : '2021 - 11 - 05',
def getcomparisonpairs() : """Return meaningful stock comparison pairs for analysis""" return [('AAPL', 'MSFT'), ('TSLA', 'AMZN'),
heavymarket('XOM', 'CVX'), Energymajors]
def getsectorrepresentatives() : """Return nonrepresentative stock per major sector""" return 'Technology' : 'AAPL', 'Energy' : 'XOM', 'Heavy' : 'CVX'

```

C.2.2 Task 2.2: Create Comprehensive Evaluation Runner

Create `comprehensive_evaluation.py`:

```

[style=pythonstyle, caption=Multi-Asset Evaluation Framework] !/usr/bin/env python3 """
Comprehensive Multi-Asset Evaluation Framework =====
"""
import pandas as pd import numpy as np from typing import Dict, List, Tuple from concurrent.futures import ThreadPoolExecutor, as_completed import logging from stock_groups import STOCKGROUPS, MARKETCONDITIONS

```

```

class ComprehensiveEvaluator: """Runs systematic evaluation across multiple assets and con-
ditions"""
    def init(self, models: Dict, config: Dict): self.models = models self.config = config self.results =
    def run_full_evaluation(self) : """Run complete evaluation across all stocks and conditions"""
        1. Individual stock evaluation self.evaluate_by_stock_groups()
        2. Market condition analysis self.evaluate_by_market_conditions()
        3. Cross-sectional analysis self.evaluate_cross_sectional()
        4. Pair trading analysis self.evaluate_pair_trading()
        return self.results
    def evaluate_by_stock_groups(self) : """Evaluate model performance by stock sector / group"""
        for group_name, group_info in STOCK_GROUPS.items() : print(f"
nEvaluating group {group_name} upper() sector...")
            group_results = symbols = group_info['symbols']
            Parallel evaluation of stocks in group with ThreadPoolExecutor(max_workers = 4) as executor :
            future_to_symbol = executor.submit(self.evaluate_single_stock, symbol) : symbol for symbol in symbols
            for future in as_completed(future_to_symbol) : symbol = future_to_symbol[future] try : result =
            future.result() group_results[symbol] = result except Exception as exc : print(f"symbol generated exception :
            exc')
            self.results[f'group_{group_name}'] = group_results
    def evaluate_by_market_conditions(self) : """Evaluate model performance under different market conditions"""
        for condition_name, condition_info in MARKET_CONDITIONS.items() : print(f"
nEvaluating condition {condition_name} period...")
            Filter data by date range start_date = condition_info['start_date'] end_date = condition_info['end_date']
            condition_results =
            Test representative stocks in this period test_symbols = ['AAPL', 'JPM', 'SPY'] Representative cross-
            section
            for symbol in test_symbols : result = self.evaluate_single_stock(symbol, start_date = start_date, end_date =
            end_date) condition_results[symbol] = result
            self.results[f'condition_{condition_name}'] = condition_results
    def evaluate_single_stock(self, symbol : str, start_date = None, end_date = None) : """Evaluate all modelson a single
            Load data for symbol with date filters data_loader = self.get_data_loader(symbol, start_date, end_date) X_train, y_train =
            data_loader.get_train_test_split()
            stock_results =
            for model_name, model in self.models.items() : try : Train model model.fit(X_train, y_train)
            Generate predictions y_pred = model.predict(X_test)
            Calculate metrics metrics = self.calculate_metrics(y_test, y_pred)
            Calculate trading performance trading_metrics = self.calculate_trading_metrics(y_test, y_pred, symbol)
            stock_results[model_name] = 'prediction_metrics' : metrics, 'trading_metrics' : trading_metrics
            except Exception as e: print(f"Error with model {model_name} on symbol : {symbol}") stock_results[model_name] =
            'error' : str(e)
            return stock_results

```

C.3 Priority 3: Statistical Analysis Framework (Days 22-28)

C.3.1 Task 3.1: Create Statistical Testing Suite

Create `statistical_analysis.py`:

```
[style=pythonstyle, caption=Statistical Analysis Framework] !/usr/bin/env python3 """ Statis-
tical Analysis and Significance Testing =====
"""

import numpy as np import pandas as pd from scipy import stats from scipy.stats import
friedmanchisquare import scikit_posthocs as spf from typing import Dict, List, Tuple

class StatisticalAnalyzer: """Comprehensive statistical analysis of model performance"""

    def __init__(self, results: Dict): self.results = results self.analysis_results = {}

    def run_full_analysis(self): """Run complete statistical analysis"""
        1. Descriptive statistics self.calculate_descriptive_stats()
        2. Significance testing self.run_significance_tests()
        3. Effect size analysis self.calculate_effect_sizes()
        4. Risk analysis self.analyze_risk_metrics()
        return self.analysis_results

    def calculate_descriptive_stats(self): """Calculate descriptive statistics for all models"""
        model_performance = self.aggregate_model_performance()
        desc_stats = {}
        for metric in ['mse', 'mae', 'sharpe_ratio', 'max_drawdown']: desc_stats[metric] = {}
        for model_name in model_performance.keys(): values = model_performance[model_name][metric]
        desc_stats[metric][model_name] = {'mean': np.mean(values), 'std': np.std(values), 'median': np.median(values)}
        self.analysis_results['descriptive_stats'] = desc_stats

    def run_significance_tests(self): """Run statistical significance tests comparing models"""
        model_performance = self.aggregate_model_performance() significance_results = {}
        for metric in ['mse', 'mae', 'sharpe_ratio']: print(f"
nTesting significance for metric...")
        Prepare data for Friedman test model_names = list(model_performance.keys()) data_matrix = []
        for model_name in model_names: data_matrix.append(model_performance[model_name][metric])
        Friedman test (non-parametric ANOVA for repeated measures) statistic, p_value = friedmanchisquare(*data_matrix)
        significance_results[metric] = {'friedman_statistic': statistic, 'p_value': p_value, 'significant': p_value < 0.05}
        Post-hoc analysis if significant if p_value < 0.05: df = pd.DataFrame(data_matrix, index =
model_names).T posthoc = spf.posthoc_nemenyi_friedman(df) significance_results[metric]['posthoc'] =
posthoc

        self.analysis_results['significance_tests'] = significance_results

    def calculate_effect_sizes(self): """Calculate effect sizes for model comparisons"""
        model_performance = self.aggregate_model_performance() effect_sizes = {}
        Cohen's d for pairwise comparisons model_names = list(model_performance.keys())
        for metric in ['mse', 'mae', 'sharpe_ratio']: effect_sizes[metric] = {}
        for i, model1 in enumerate(model_names): for j, model2 in enumerate(model_names[i+1:], i+1):
        values1 = model_performance[model1][metric] values2 = model_performance[model2][metric]
```

```

Cohen's d pooled_std = np.sqrt(((len(values1) - 1) * np.var(values1) + (len(values2) - 1) *
np.var(values2)) / (len(values1) + len(values2) - 2))
cohens_d = (np.mean(values1) - np.mean(values2)) / pooled_std
effect_sizes[metric][f'model1_vs_model2'] = 'cohens_d : cohens_d, 'magnitude' : self.interpret_cohens_d(cohens_d)
self.analysis_results[effect_sizes] = effect_sizes
@staticmethod def interpret_cohens_d(d) : """ InterpretCohen's d effect size """ abs_d = abs(d) if abs_d <
0.2 : return 'negligible' elif abs_d < 0.5 : return 'small' elif abs_d < 0.8 : return 'medium' else :
return 'large'

```

C.4 Priority 4: Paper Generation Framework (Days 26-31)

C.4.1 Task 4.1: Automated Results Table Generation

Create `paper_generator.py`:

```

[style=pythonstyle, caption=Automated Paper Generation] !/usr/bin/env python3 """ Auto-
mated Research Paper Content Generation =====
"""

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from typing import Dict, Any

class PaperGenerator: """Generate LaTeX tables and figures for research paper"""
    def __init__(self, results: Dict, analysis: Dict):
        self.results = results
        self.analysis = analysis

    def generate_main_results_table(self) : """Generate main results comparison table"""
        latex_table = r"""
\begin{table}[htbp]
\centering
\caption{Model Performance Comparison Across All Stocks}
\label{tab : main_results}
\begin{tbl_struct}
\newcolumntype{r}{ccccc}
\toprule
Model & MAE & RMSE & Directional Accuracy & Sharpe Ratio & Max Drawdown & Win Rate
\midrule
"""

        # Extract aggregated results for model_name in self.get_model_names() : metrics = self.get_aggregated_metrics(model_name)
        latex_table += f"model_name" latex_table += f"metrics['mae'] : .4f" latex_table += f"metrics['rmse'] : .4f" latex
        latex_table += f"metrics['directional_accuracy'] : .3f" latex_table += f"metrics['sharpe_ratio'] : .3f" latex_table += f
        latex_table += f"metrics['max_drawdown'] : .3f" latex_table += f"metrics['win_rate'] : .3f"

        latex_table += r"""
\bottomrule
"""

```

```

endtabular
endtable"""

    return latex_table

    def generate_sector_analysis_table(self): """Generatesector – wiseperformanceanalysis"""
    latex_table = r"""
begin table[htbp]
centering
caption TFT Performance by Market Sector
label tab : sector_analysis
begin tabular lcccc
toprule Sector Sharpe Ratio Max Drawdown Total Return Volatility
Stocks

midrule"""

    for sector in ['Technology', 'Financial', 'Healthcare', 'Consumer', 'Energy']: sector_metrics =
self.get_sector_metrics(sector)
        latex_table += f"sector" latex_table += f"sector_metrics['sharpe_ratio'] : .3f" latex_table += f"sector_metrics['m
f"sector_metrics['total_return'] : .3f" latex_table += f"sector_metrics['volatility'] : .3f" latex_table +=
f"sector_metrics['num_stocks']

    latex_table += r"""
bottomrule
endtabular
endtable"""

    return latex_table

    def generate_significance_table(self): """Generate statistical significance result stable"""
    significance = self.analysis['significance_tests']
    latex_table = r"""
begin table[htbp]
centering
caption Statistical Significance Tests (Friedman Test)
label tab : significance
begin tabular lccc
toprule Metric Test Statistic p – value Significant

midrule"""

```



```

        for metric, results in significance.items(): latex_table+= f"metric.upper()" latex_table+= f"results['friedman'
f"results['p_value'] : .4f" latex_table+= f"'Yes' if results['significant'] else 'No'

    """
    latex_table+= r"""
bottomrule
endtabular
endtable"""
    return latex_table
def generate_all_tables(self): """Generate all tables for the paper"""
    tables = 'main_results' : self.generate_main_results_table(), 'sector_analysis' : self.generate_sector_analysis_table()
self.generate_significance_table()
    return tables

```

D Daily Task Breakdown

D.1 Week 1: Baseline Implementation

Day 1-2: LSTM and Deep Learning Baselines

- Implement LSTM variants (vanilla, bidirectional, stacked)
- Add GRU models and CNN-LSTM hybrid
- Integrate with existing `fixed_baseline_pipeline.py`

Day 3-4: Traditional ML Enhancement

- Add LightGBM, CatBoost, Extra Trees
- Implement SVM with different kernels
- Add ensemble methods (voting, stacking)

Day 5-6: Financial Time Series Models

- Implement ARIMA, GARCH, VAR models
- Add Markov switching models
- Create naive baselines (random walk, moving averages)

Day 7: Testing and Integration

- Test all baseline models
- Fix integration issues
- Validate output consistency

D.2 Week 2-3: Multi-Asset Evaluation

Day 8-10: Stock Group Analysis

- Implement `stock_groups.py`
- Run technology sector analysis
- Run financial sector analysis

Day 11-14: Comprehensive Stock Coverage

- Healthcare and consumer sectors
- Energy and utilities sectors
- Index ETF analysis

Day 15-17: Market Condition Analysis

- Bull market periods (2020-2021)
- Bear market periods (2022)
- Volatile periods (2023-2024)

Day 18-21: Cross-sectional Analysis

- Pair trading analysis
- Correlation studies
- Portfolio-level evaluation

D.3 Week 4: Statistical Analysis

Day 22-24: Core Statistical Testing

- Implement `statistical_analysis.py`
- Run Friedman tests for model comparison
- Calculate effect sizes (Cohen's d)

Day 25-26: Advanced Analytics

- Risk-adjusted performance metrics
- Regime-dependent analysis
- Robustness testing

Day 27-28: Results Compilation

- Aggregate all results
- Generate summary statistics
- Create visualization pipeline

D.4 Week 5: Paper Writing

Day 29-30: Content Generation

- Implement `paper_generator.py`
- Generate all LaTeX tables
- Create figures and plots

Day 31: Final Assembly

- Compile complete paper
- Final review and editing
- Submission preparation

E Success Metrics

E.1 Technical Deliverables

- 15+ baseline models implemented and tested
- 25+ stocks analyzed across 6 sectors
- 3+ market conditions evaluated
- Statistical significance established
- Complete paper with tables and figures

E.2 Research Quality

- Rigorous statistical methodology
- Comprehensive baseline comparison
- Multi-dimensional evaluation
- Reproducible results
- Clear contribution statement

Implementation roadmap created: July 17, 2025